

Lecture 3

CMPE 240

Number Systems and Codes

Alper Şen

Bogazici University

Overview

- Chapter 2 from Wakerly
- Number Systems (Conversions)
 - Binary
 - Octal
 - Hex
- Negative Numbers
 - Signed Magnitude
 - Two's Complement
- Arithmetic
 - Addition (Subtraction)
 - Multiplication
- Codes

Positional Number Systems

- The traditional number system is called a positional number system.

$$6354 = 6 * 1000 + 3 * 100 + 5 * 10 + 4$$

- A number is represented as a string of digits.
- Each digit position has a *weight* assoc. with it.
- Number's value = a *weighted sum* of the digits
- Positive and negative weights

$$D = \sum_{i=0}^{p-1} d_i 10^i$$

Fractions: Weights that are Negative Powers of 10

$$425.97 = 4 * 10^2 + 2 * 10^1 + 5 * 10^0 + 9 * 10^{-1} + 7 * 10^{-2}$$

$$D = \sum_{i=-n}^{p-1} d_i 10^i$$

Binary Numbers

$$B = \sum_{i=-n}^{p-1} b_i 2^i \quad \dots b_1 2^1 + b_0 2^0 + b_{-1} 2^{-1}$$

100101.0011

- The “**base**” (**radix**) is 2 instead of 10
- Meaning: the weights are powers of 2 instead of powers of 10.
- Digits are called “bits,” for “binary digits.”

Octal and Hexadecimal (“Hex”) Numbers

- Octal = base 8
- Hexadecimal = base 16
 - Use A – F to represent the values 10 through 15 in each position.

Decimal	Binary	Octal	Hex
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

Usefulness of Octal and Hex Numbers

- Useful for representing multibit binary numbers because their radices are integer multiples of 2.
- In octal, groups of 3 binary numbers and in hex, groups of 4 binary numbers are encoded.
- When converting from binary, pad left hand side and right hand side with zeros as required.

Binary to HEX Example:

10 0101 1010 **1111** . **1011** 111₂ =

0010 0101 1010 **1111** . **1011** 111**0**₂ =

2 5 A **F** . **B** E₁₆

Decimal-to-Radix-r Conversions

- Radix-r-to-decimal conversions are easy since we do arithmetic in decimal.
 - $(111001)_2 = 1 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 57$
- However, decimal-to-radix-r conversions using decimal arithmetic is harder.
- To do the latter conversion, we convert the integer and fractional parts separately and add the results afterwards.

Decimal-to-Radix-r Conversions: Integer Part

- Successively divide number by r , taking remainder as result. **Why?** Use representation $\dots b_1 2^1 + b_0 2^0$

$$b_2 2^2 + b_1 2^1 + b_0 2^0$$

=

$$2 (b_2 2^1 + b_1 2^0) + b_0 \quad \text{divide by 2}$$

$$2 (b_2 2^0) + b_1 \quad \text{divide by 2}$$

$$2(0) + b_2$$

Decimal-to-Radix-r Conversions: Integer Part

- Successively divide number by r , taking remainder as result.
- Example: Convert 57_{10} to binary.

$57 / 2 = 28$ remainder $1 = b_0$ (Least Significant Bit - LSB)

$/2 = 14$ remainder $0 = b_1$

$/2 = 7$ remainder 0

$/2 = 3$ remainder 1

$/2 = 1$ remainder 1

$/2 = 0$ remainder 1 (MostSignifBit-MSB)

Ans: 111001_2

Decimal-to-Radix-r Conversions:

Fractional Part

- Successively multiply number by r , taking integer part as result and chopping off integer part before next iteration.

Why?

Use representation $b_{-1}2^{-1} + b_{-2}2^{-2} + b_{-3}2^{-3}$

- May be unending!

$$(b_{-1}2^{-1} + b_{-2}2^{-2} + b_{-3}2^{-3})$$

$$2(b_{-1}2^{-1} + b_{-2}2^{-2} + b_{-3}2^{-3}) \quad \text{multiply by 2}$$

$$= b_{-1} + (b_{-2}2^{-1} + b_{-3}2^{-2}) \quad \text{integer and fraction parts}$$

$$2(b_{-2}2^{-1} + b_{-3}2^{-2}) \quad \text{multiply by 2}$$

$$= b_{-2} + b_{-3}2^{-1} \quad \text{integer and fraction parts}$$

$$2(b_{-3}2^{-1}) \quad \text{multiply by 2}$$

$$= b_{-3} \quad \text{integer and no fraction part}$$

Decimal-to-Radix-r Conversions:

Fractional Part

- Successively multiply number by r , taking integer part as result and chopping off integer part before next iteration.
Why? Hint: Use representation $b_{-1}2^{-1} + b_{-2}2^{-2} + b_{-3}2^{-3}$
- May be unending!
- Example: Convert $.3_{10}$ to binary.

$$.3 * 2 = 0.6 \quad \text{integer part} = 0 = b_{-1} \text{ (LSB)}$$

$$.6 * 2 = 1.2 \quad \text{integer part} = 1 = b_{-2}$$

$$.2 * 2 = 0.4 \quad \text{integer part} = 0 = b_{-3}$$

$$.4 * 2 = 0.8 \quad \text{integer part} = 0 = b_{-4}$$

$$.8 * 2 = 1.6 \quad \text{integer part} = 1 = b_{-5}$$

$$.6 * 2 = 1.2 \quad \text{integer part} = 1, \text{ etc. unending}$$

$$\text{Ans} = .01001$$

Conversions

$$13.375_{10} = ?_2$$

13 / 2 = 6 R 1

6 / 2 = 3 R 0

3 / 2 = 1 R 1

1 / 2 = 0 R 1 (MSB)

1101

$$13.375_{10} = 1101.011_2$$

Wakerly Section 2.4 and further

Addition and Subtraction of
Nondecimal Numbers

Addition and Subtraction

- Use same technique as decimal addition and subtraction
- Except that the addition and subtraction tables are different as shown next
- **a carry is a digit that is transferred from one column of digits to another column of more significant digits during addition.**
- When used in subtraction the operation is called a **borrow**.
- Use Carry_{in} and $\text{Carry}_{\text{out}}$ for addition
- Use $\text{Borrow}_{\text{in}}$ and $\text{Borrow}_{\text{out}}$ for subtraction
- Formalizing Carry/Borrow helps later on for building circuit, checking overflow etc.

Addition and Subtraction Table for Binary System

Inputs

Addition Outputs

Subtraction Outputs

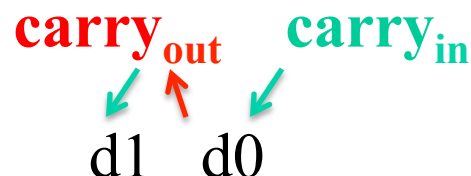
carry_{in} or borrow_{in}	x	y	carry_{out}	sum	borrow_{out}	difference
0	0	0	0	0	0	0
0	0	1	0	1	1	1
0	1	0	0	1	0	1
0	1	1	1	0	0	0
1	0	0	0	1	1	1
1	0	1	1	0	1	0
1	1	0	1	0	0	0
1	1	1	1	1	1	1

Examples

- Carry out of a digit is the carry in of the next higher digit

- $x + y = 190 + 141 = 331$

- $$\begin{array}{r} 1\ 0\ 1\ 1\ 1\ 1\ 0\ 0\ 0\ (\text{carry}) \\ 1\ 0\ 1\ 1\ 1\ 1\ 1\ 0\ (\text{x})\ \text{addend} \\ +\ 1\ 0\ 0\ 0\ 1\ 1\ 0\ 1\ (\text{y})\ \text{addend} \\ \hline 1\ 0\ 1\ 0\ 0\ 1\ 0\ 1\ 1\ (\text{sum}) \end{array}$$



- $229 - 46 = 183$

- $$\begin{array}{r} 0\ 0\ 1\ 1\ 1\ 1\ 1\ 0\ 0\ (\text{borrow}) \\ 1\ 1\ 1\ 0\ 0\ 1\ 0\ 1\ (\text{x})\ \text{minuend} \\ -\ 0\ 0\ 1\ 0\ 1\ 1\ 1\ 0\ (\text{y})\ \text{subtrahend} \\ \hline 1\ 0\ 1\ 1\ 0\ 1\ 1\ 1\ (\text{difference}) \end{array}$$

- Note: If x-y operation generates borrow out of the most significant bit position then x is less than y.

Addition and Subtraction of Octal and Hexadecimal Numbers

- Not really too different
- **Exercise:** Develop the addition and subtraction table for hexadecimal numbers.

Section 2.5: Representation of Negative Numbers

- More accurately: representation of signed numbers
 - a. Signed-magnitude representation
 - b. Radix-complement representation
 - **2's-complement representation (What we use)**
 - c. Diminished radix-complement representation
 - 1's complement representation

A. Signed-magnitude representation

- Also called, “sign-and-magnitude representation”
- A number consists of a **magnitude** and a **sign** bit
- Usually 0 in sign bit means positive, 1 negative
 - Usually the entire number is represented with a single sign bit to the left, followed by a number of magnitude bits

A. Signed-magnitude representation

+10_{ten}
001010_{two}
sign bit

-10_{ten}
101010_{two}
sign bit

- The number zero has two representations (+0 and -0) 0000 and 1000, respectively.
- An n-bit signed-magnitude number lies within the range $-(2^{n-1} - 1)$ *through* $+(2^{n-1} - 1)$
- **Example:** Given n=3, then numbers are -3 -2 -1 -0 0 1 2 3
- To add signed-magnitude numbers we must examine the signs of the addends to determine what to do ...

Representations of Negative Numbers

<i>Decimal</i>	<i>Signed Magnitude</i>
—	—
—8	—
—7	1111
—6	1110
—5	1101
—4	1100
—3	1011
—2	1010
—1	1001
0	1000 or 0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111

Positive numbers
have the same
representation
for 2' s comp,
1' s comp,
signed mag.

Machine arithmetic with signed-magnitude representation

- Takes several steps to add a pair of numbers
 - Examine signs of the addends
 - If the signs are the same, then add magnitudes and give the result the same sign as the operands
 - If different, must...
 - Compare magnitude of the two operands
 - Subtract smaller number from larger
 - Give the result the sign of the larger operand
 - Ex: $85 + (-127)$
- For this reason the signed-mag representation is not as popular in computers as one might think

Complement Number Systems

- Negates a number by *taking its complement* instead of negating the sign $1000 \rightarrow 0111$
- Exact meaning of *taking its complement* is defined in various ways – will see
- Not natural for humans, but better for machine arithmetic
- Will describe 2 complement number systems
 - *Radix complement – very popular in real computers*
 - *Diminished radix-complement*

B. Radix-complement number representation

- First decide how many bits to represent the number— say n .

- Radix complement of a number is defined as

$r^n - \text{number}$

flipping the digits

$= ((r^n - 1) - \text{number}) + 1$ (this representation avoids subtraction)

- Example: $n=4$ -bit decimal, 0000 to 9999:

– Original number = 3524

flipping the digits

– 10's complement = $((10^4 - 1) - 3524) + 1 = (9999 - 3524) + 1 = 6476$

- 10's complement

0 and Positive numbers: 0000-4999

Negative numbers: 5000-9999, where 9999 is 'minus 1.'

Note that, there is no "negative" 0 and there is an extra negative number 5000, (10's complement of 5000 is 5000)

2's complement representation

- Just radix-complement when radix = 2
- Used a lot in computers and other digital arithmetic circuits
- 0 and positive numbers have the leftmost (MSB) bit = 0
- negative numbers have the leftmost (MSB) bit = 1
- To find a number's 2's complement – just flip all the bits and add 1
– $= ((r^n - 1) - \text{number}) + 1 = (111..1_2 - \text{number}) + 1$
- An n-bit 2's complement number lies within the range $-(2^{n-1})$ through $+(2^{n-1} - 1)$ (*extra negative number, cannot represent positive 2^{n-1} not enough bits*)
- Example: n=4 bits can represent numbers from -8 to +7
- +7 = 0111, +8 = 01000 !!! Need 5 bits not possible with 4 bits
- -7 = 1001 -8 = 1000

Representations of Negative Numbers

<i>Decimal</i>	<i>Two's Complement</i>	<i>Ones' Complement</i>	<i>Signed Magnitude</i>
−8	1000	—	—
−7	1001	1000	1111
−6	1010	1001	1110
−5	1011	1010	1101
−4	1100	1011	1100
−3	1101	1100	1011
−2	1110	1101	1010
−1	1111	1110	1001
0	0000	1111 or 0000	1000 or 0000
1	0001	0001	0001
2	0010	0010	0010
3	0011	0011	0011
4	0100	0100	0100
5	0101	0101	0101
6	0110	0110	0110
7	0111	0111	0111

Positive numbers
have the same
representation
for 2' s comp,
1' s comp,
signed mag.

Examples: 2's complement (4-bit)

$$\begin{array}{ccc} +6_{10} = 0110_2 & \xrightarrow{\text{blue arrow}} & 1010_2 = -6_{10} \\ \downarrow \text{green arrow} & & \downarrow \text{blue arrow} \\ 1001 & & 0110_2 = +6_{10} \\ +1 & & \\ \hline 1010_2 = -6_{10} & & \\ \downarrow \text{green arrow} & & \\ 0101 & & \\ +1 & & \\ \hline 0110_2 = +6_{10} & & \end{array}$$

2's comp to decimal conversion is the same as unsigned base 2 to decimal conversion, except weight of the MSB is -2^{n-1}

C. Diminished Radix

Complement: 1's complement

- To find a number's 1's complement – just flip all the bits
- $0110 \rightarrow 1001$
- The number zero has two representations (+0 and -0)
- $+0 = 0000$ and $-0 = 1111$

Representations of Negative Numbers

<i>Decimal</i>	<i>Two's Complement</i>	<i>Ones' Complement</i>	<i>Signed Magnitude</i>
−8	1000	—	—
−7	1001	1000	1111
−6	1010	1001	1110
−5	1011	1010	1101
−4	1100	1011	1100
−3	1101	1100	1011
−2	1110	1101	1010
−1	1111	1110	1001
0	0000	1111 or 0000	1000 or 0000
1	0001	0001	0001
2	0010	0010	0010
3	0011	0011	0011
4	0100	0100	0100
5	0101	0101	0101
6	0110	0110	0110
7	0111	0111	0111

Positive numbers
have the same
representation
for 2' s comp,
1' s comp,
signed mag.

2's Complement Examples

the max positive representable with 8-bits is +127

$n=8$

$$\begin{array}{r} 17_{10} = 00010001_2 \\ \downarrow \text{complement} \\ 11101110 + \\ \hline 11101111_2 = -17_{10} \end{array}$$

$$\begin{array}{r} 127_{10} = 01111111 \\ \downarrow \text{complement} \\ 10000000 + \\ \hline 10000001_2 = -127_{10} \end{array}$$

$$\begin{array}{r} -128_{10} = 10000000_2 \\ \downarrow \text{complement} \\ 01111111 + \\ \hline 10000000 = -128_{10} \end{array}$$

$$\begin{array}{r} 0_{10} = 00000000 \\ \downarrow \text{complement} \\ 11111111 + \\ \hline (1)00000000 = 0_{10} \end{array}$$

2's Complement Addition and Subtraction Rules

- Addition is just an extension of ordinary counting
- For $n=4$, starting from 1000 (-8) on up, each successive 2's comp number all the way to 0111 (+7) can be obtained by adding 1 to the previous one, **ignoring any carries beyond the 4th bit position** (the same argument not true for sign magnitude or 1's complement)
- Advantage over signed magnitude:
 - Since addition is just an extension of ordinary counting, 2's comp numbers can be added by ordinary binary addition
 - No different cases based on operands' signs!

2's Complement Addition

$$\text{for } n=4 \Rightarrow 2^{n-1} = 8$$

1. Addition of two positive numbers, $\text{sum} < 2^{n-1}$

+3	0011	
+4	0100	
+7	0111	(correct answer)

2. Addition of two positive numbers, $\text{sum} \geq 2^{n-1}$

+5	0101	(Carry bit into sign bit position is different from the carry bit out of the sign bit position)
+6	0110	
+11	1011	← wrong answer because of overflow (+11 requires 5 bits including sign)

2's Complement Addition

$$\text{for } n=4 \Rightarrow 2^{n-1} = 8$$

3. Addition of positive and negative numbers
(negative number has greater magnitude).

$$\begin{array}{r} +5 \quad 0101 \\ -6 \quad 1010 \\ \hline -1 \quad 1111 \text{ (correct answer)} \end{array}$$

4. Addition of positive and negative numbers
(positive number has greater magnitude).

$$\begin{array}{r} -5 \quad 1011 \\ +6 \quad 0110 \\ \hline \end{array}$$

+1 (1) 0001 correct answer when carry from sign bit is ignored
(not an overflow)

2's Complement Addition

$$\text{for } n=4 \Rightarrow 2^{n-1} = 8$$

5. Addition of two negative numbers, $|\text{sum}| \leq 2^{n-1}$.

-3 1101

-4 1100

=====

-7 (1) 1001 correct answer when carry from sign bit is
ignored (not an overflow)

6. Addition of two negative numbers, $|\text{sum}| > 2^{n-1}$.

-5 1011

-6 1010

=====

(Carry bit into sign bit position is different from
the carry bit out of the sign bit position)

-11 (1) 0101 wrong answer because of overflow (-11
requires 5 bits including sign)

2's Complement Subtraction

- Negate the subtrahend by taking its 2's complement and then add it to the minuend using the normal rules for addition.
- $+4 - (+3) = +1$
- $0100 - 0011$
- $+4 + (-3)$
- $0100 + 1101 = (1) 0001$ (ignore carry, no overflow)
- **An adder circuit and a circuit for obtaining 2's complement is sufficient to do subtraction**

Overflow in Signed Arithmetic

- Operation produces a result that exceeds the number system
- Example (4-bits)
 - Range -8 to +7

Detecting Overflow

- Overflow occurs when the value affects the sign bit:
 - adding two positives yields a negative
 - adding two negatives gives a positive
 - subtract a negative from a positive and get a negative
 - subtract a positive from a negative and get a positive
- No overflow when adding a positive and a negative number
- No overflow when subtracting two numbers of same sign
- Consider the operations $A + B$, and $A - B$
 - Can overflow occur if B is 0 ? **cannot occur !**
 - Can overflow occur if A is 0 ? **can occur !**
(for $A - B$ if $B = -2^{n-1}$)

Overflow in Addition

- The two's complement of the minimum number in the range will not have the desired effect of negating the number.
 - For example, the two's complement of -128 in an 8-bit system results in the same binary number. This is because a positive value of 128 cannot be represented with an 8-bit signed binary number.
- **Overflow Detection Rule for addition**
 - Overflow occurs if the addends' signs are the same, but the sum's sign is different from the addends'
- **Note that this can be detected by a circuit as an overflow condition since there was a carry into but not out of the sign bit**
- $(+5) + (+6) = +11$
- $0101 + 0110 = 1011$ Overflow!

Overflow in Subtraction

- Overflow can be detected by checking the signs of numbers being added (or by checking carry in and out of sign position **except when the subtrahend is the most negative number**).
- $+2 - (-8) = 0010 + 1000 = 1010 = -6$
- No overflow is detected since carry bit sign position is the same as carry bit out of sign bit; however the result is incorrect.

2's Complement Sign Extension

- As far as $m > n$, it is possible to convert n -bit numbers into m -bit numbers, but some care is needed :
 - copy the most significant bit (the sign bit) into the other bits
(Example: 4 bit to 8 bit)

0010 $\rightarrow$ 0000 0010 +2

1010 $\rightarrow$ 1111 1010 -6

- This procedure is referred as "sign extension"
- **Question:** What about if $m < n$?

1's Complement Addition

- Perform a standard binary addition; if there is a carry out of the sign position add 1 to the result.
(end-around carry rule)

- $-2 + -5 = -7$, $1101 + 1010 = (1) 0111 \quad + 1 = 1000$

- $-0 + (-0) = -0$, $1111 + 1111 = (1) 1110 \quad + 1 = 1111$

Advantages of 2's Complement number system

- The 2's-complement system has the advantage of not requiring that the addition and subtraction circuitry **examine the signs of the operands** to determine whether to add or subtract (as in signed magnitude system).
- This property makes the system both simpler to implement and capable of easily handling higher precision arithmetic.
- Also, zero has only a single representation, obviating the subtleties associated with negative zero, which exists in 1's-complement system, and signed magnitude system.
- The problems of **multiple representations of 0 and the need for the end-around carry** are circumvented in 2's complement system.

Binary multiplication

- Grammar school method for decimal: add a list of shifted multiplicands computed according to the digits of the multiplier
- Same method can be used in binary
- For two unsigned operands, the only possible values of the multiplier digits are 0 and 1
 - Thus it's trivial to form the shifted multiplicands

Binary Multiplication

X	multiplicand				X_3	X_2	X_1	X_0
$\times Y$	multiplier				$\times Y_3$	Y_2	Y_1	Y_0
P					X_3Y_0	X_2Y_0	X_1Y_0	X_0Y_0
				X_3Y_1	X_2Y_1	X_1Y_1	X_0Y_1	
		X_3Y_2	X_2Y_2	X_1Y_2	X_0Y_2			
	$+$	X_3Y_3	X_2Y_3	X_1Y_3	X_0Y_3			
		P_7	P_6	P_5	P_4	P_3	P_2	P_1
								P_0

Multiplication Example

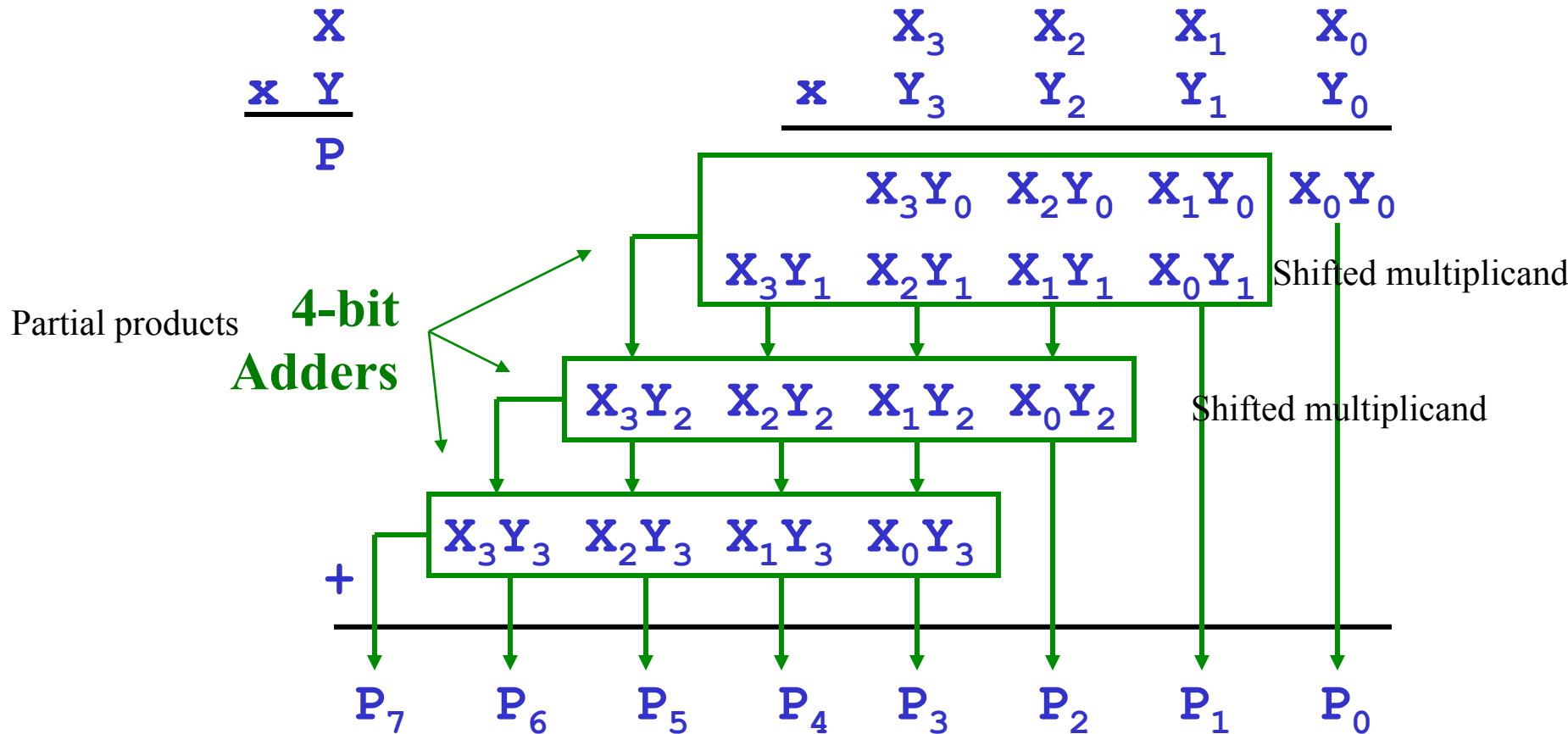
$$\begin{array}{r} 7 \\ \times 5 \\ \hline 35 \end{array}$$

$$\begin{array}{r} 0 \ 1 \ 1 \ 1 \\ \times 0 \ 1 \ 0 \ 1 \\ \hline 0 \ 1 \ 1 \ 1 \\ 0 \ 0 \ 0 \ 0 \\ 0 \ 1 \ 1 \ 1 \\ + 0 \ 0 \ 0 \ 0 \\ \hline 0 \ 0 \ 1 \ 0 \ 0 \ 0 \ 1 \ 1 \end{array}$$

Binary multiplication in binary on a machine

- Instead of listing all shifted multiplicands and then adding, it is more convenient to add each shifted multiplicand as it is created to a partial product
- In general when we multiply an n -bit number by an m -bit number, the result requires at most $n+m$ bits to express
- The shift-and-add algorithm requires m partial products and additions to obtain result, but the 1st addition is trivial (adding to 0)

Combinational Multiplier



Binary Multiplication

- **Excercise:** Perform 2' s complement multiplication.

Codes in Computers

- BCD
- Gray code
- One-hot encoding
- Parity codes

Binary Codes for Decimal (BCD) Numbers

<i>Decimal digit</i>	<i>BCD (8421)</i>
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

unused code words

1010
1011
1100
1101
1110
1111

Each decimal digit is represented by 4 bits.

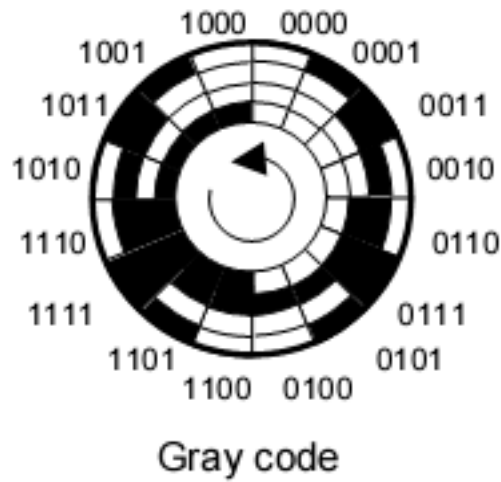
Decimal : 25

BCD: 0010 0101

Allows easy conversion to decimal digits for printing or display, and allows faster decimal calculations.

Gray Code

- The Gray code (reflected binary code) was originally designed to prevent spurious output from electromechanical switches.
- Today, Gray codes are widely used to facilitate error correction in digital communications such as digital terrestrial television and some cable TV systems.



This avoids the possibility that, when several bits change in the binary representation of an angle, a misread will result from some of the bits changing before others.

For example, 011 to 100 requires 3 synchronous changes but intermittently you can observe **011 — 001 — 101 — 100** wrong transitional states

Gray Code

<i>Decimal Number</i>	<i>Binary Code</i>	<i>Gray Code</i>
0	000	000
1	001	001
2	010	011
3	011	010
4	100	110
5	101	111
6	110	101
7	111	100

3-bit Gray code

Definition: Two successive values differ in only one bit.

Gray Code

- Rules to obtain Gray Code from a binary number
 - Bits are numbered from right to left
 - Bit i of the gray code is 0 if bits i and $i+1$ of the corresponding binary code are the same else bit i of the gray code is 1. (when $i+1 = n$, bit n of the binary code is considered to be 0)
 - Example: $011 \rightarrow 010$
 - Bit index: 210

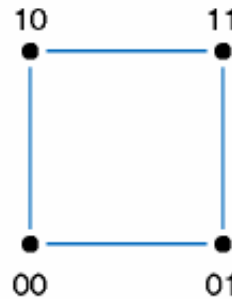
1-out-of-n (one-hot) code

- 1-out-of-3 code (**one-hot**) has 3 elements, code words
 - 100, 010, 001
- Inverted 1-out-of-3 (**one-cold**)
 - 110, 101, 011
- **m-out-of-n** code is a generalization. m bits equal to 1, rest is 0.
(2 out of 4, has 6 codes: 0011, 0101, 1001, 1100, 1010, 0110)
- Example: device selection for 8 devices
 - 1-out-of-8 code, device select uses the hot bit as enable input
 - Simple comparison but needs more bits (8 bits) than binary.
 - Binary coded select: each device compares the seen number with its own number to see if it is enabled. Fewer bits. (3 bits has 8 codes)

N-cubes and Hamming distance

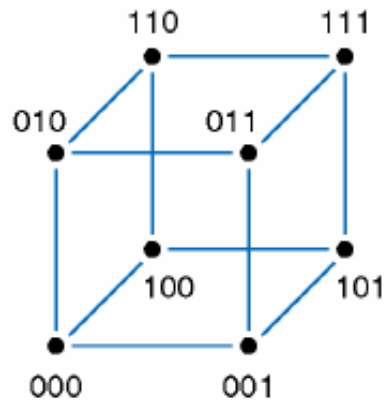


1-cube

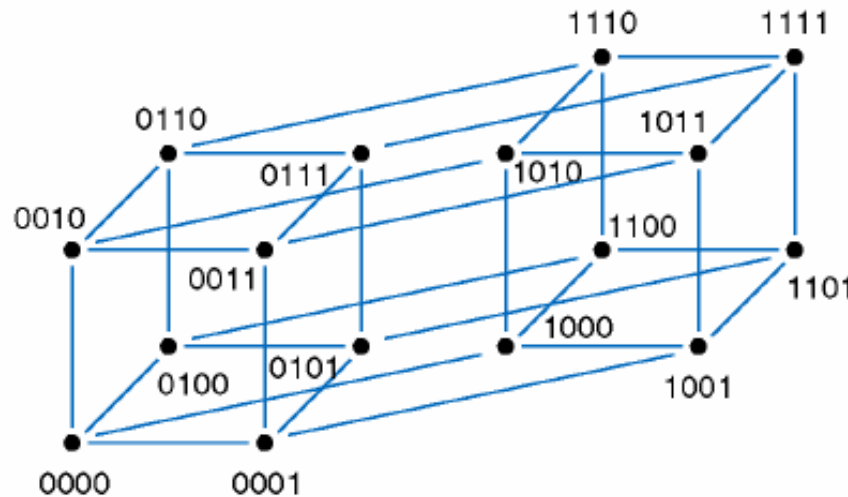


2-cube

An n -bit string can be visualized, as a vertex of an n -cube.
An n -cube has 2^n vertices.



3-cube

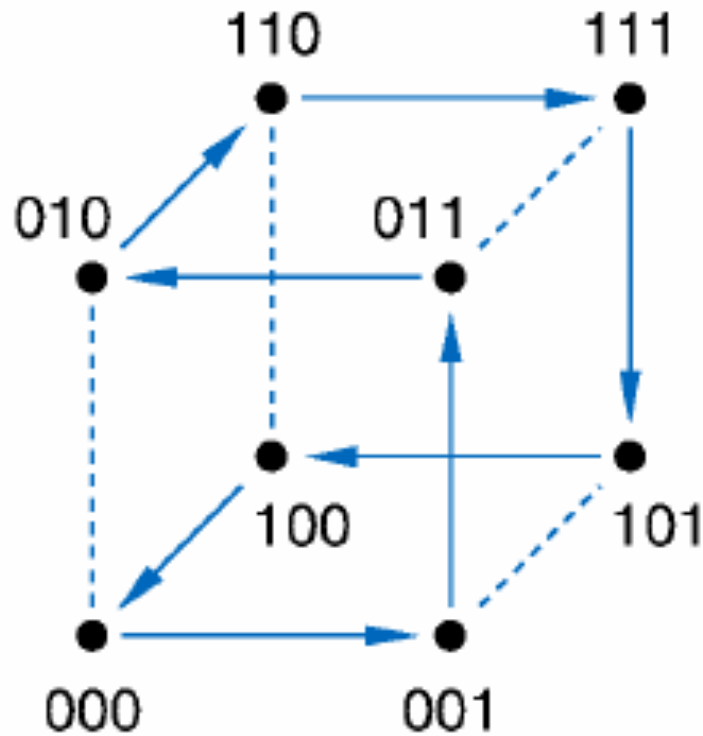


4-cube

Figure 2-8
 n -cubes for $n = 1, 2, 3$, and 4.

Hamming distance: Distance between two n -bit strings is the no. of bit positions in which they differ

Traversing a 3-cube in Gray code order



(a)

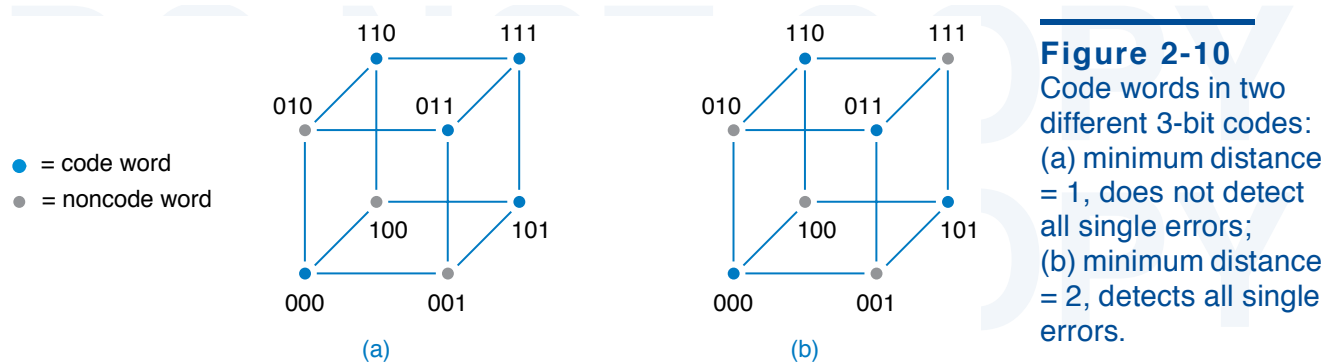
Gray code is a path that visits each vertex once

Error Detecting Codes

- An error in a digital system is the corruption of data from its correct value, caused by a temporary (cosmic ray) or permanent (heat) physical failure. (0 becomes 1 or 1 becomes 0)
- **Definition:** An error detecting code has the property that corrupting a **code word (an element of the code)** will likely produce a bit string that is not a code word.
- Hence, if the bit string is not a code word then it has an error.
- Assume a single error model. (single bit changes)

Minimum distance

- Given a code $C = \{000, 101, 011, 110, 111\}$, a single bit error in one of the code words, say in 111, may result in another code word in C (such as 110), hence C does not detect single errors.
- However, $D = \{000, 101, 011, 110\}$ detects single errors.
- A code detects all single errors if the minimum distance between all possible pairs of code words is 2.**



Parity Codes

- **Parity** codes are formed by concatenating a *parity bit*, P to each code word of C .
 - In an *odd-parity code*, the **parity** bit is specified so that the total number of ones is odd.
 - In an *even-parity code*, the **parity** bit is specified so that the total number of ones is **even**.
- Information bit: first n -bits of a code word Parity bit: the last bit
- Examples:
 - | | | |
|-------------------------|------|-----------------------|
| 1 0 0 | 1000 | 0 |
| information bits | | odd parity bit |
 - 000 1, 001 0, 010 0 ...

Parity Codes

<i>Information Bits</i>	<i>Even-parity Code</i>	<i>Odd-parity Code</i>
000	000 0	000 1
001	001 1	001 0
010	010 1	010 0
011	011 0	011 1
100	100 1	100 0
101	101 0	101 1
110	110 0	110 1
111	111 1	111 0

Parity Codes

- They result in minimum distance 2 code, hence single errors can be detected.
- Parity code does not detect 2-bit errors, since changing two bits does not affect parity, $001\ 1 \rightarrow 010\ 1$
- In fact, parity codes can detect any odd number of errors.
- Hint: odd becomes even, even becomes odd; $\text{odd} + \text{odd} = \text{even}$, $\text{odd} - \text{odd} = \text{even}$, similarly for even

Summary

- Number Systems (Conversions)
 - Binary
 - Octal
 - Hex
- Negative Numbers
 - Signed Magnitude
 - Two's Complement
- Arithmetic
 - Addition (Subtraction)
 - Multiplication
- Codes